

Chapter 9

Inheritance

C++ Inheritance

Reaccessability is yet another feature of OOP's. C++ strongly supports the concept of reusability. The C++ classes can be used again in several ways. Once a class has been written and tested, it can be adopted by other programmers. This is basically created by defining the new classes, reusing the properties of existing ones. The mechanism of deriving a new class from an old one is called **inheritance**. This is often referred to as IS-A' relationship because very object of the class being defined "is" also an object of inherited class. The old class is called **Base** class and the new one is called **Derieved** class.

Inheritance is a feature or a process in which, new classes are created from the existing classes. The new class created is called “derived class” or “child class” and the existing class is known as the “base class” or “parent class”. The derived class now is said to be inherited from the base class.

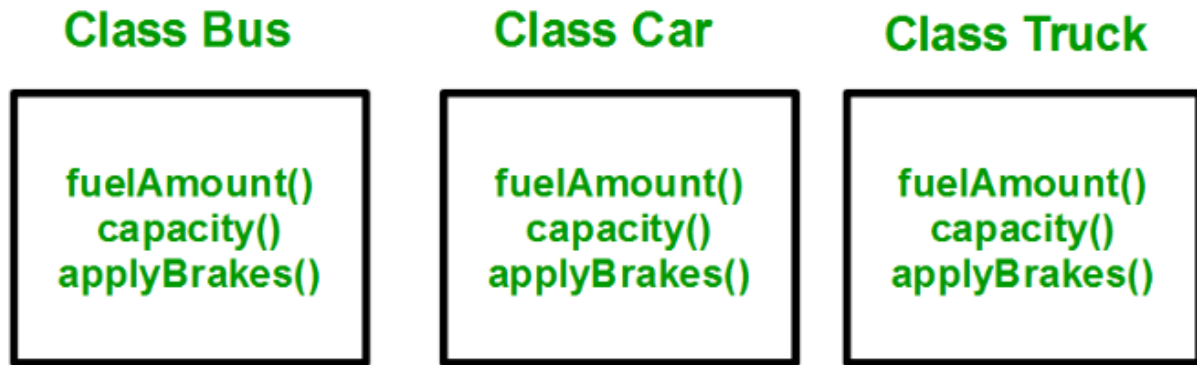
When we say derived class inherits the base class, it means, the derived class inherits all the properties of the base class, without changing the properties of base class and may add new features to its own. These new features in the derived class will not affect the base class. The derived class is the specialized class for the base class.

Advantage of C++ Inheritance

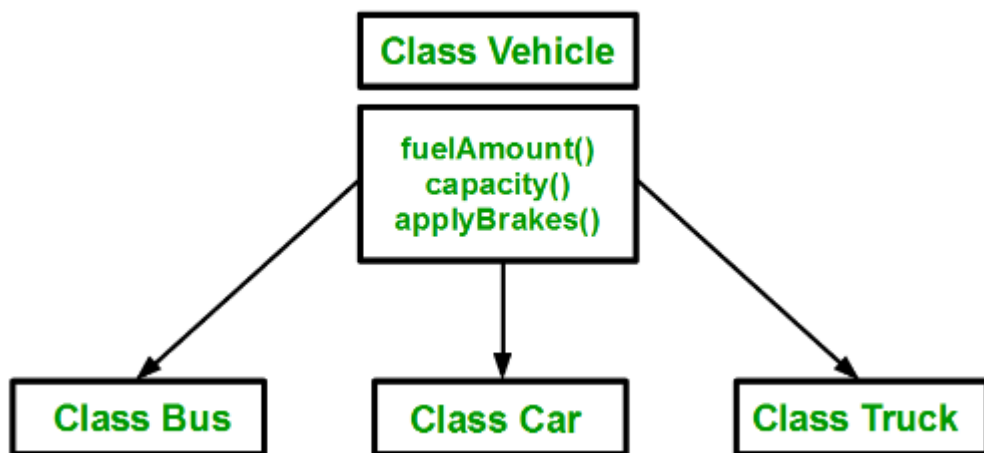
Code reusability: Now you can reuse the members of your parent class. So, there is no need to define the member again. So less code is required in the class.

Why and when to use inheritance?

Consider a group of vehicles. You need to create classes for Bus, Car, and Truck. The methods fuelAmount(), capacity(), applyBrakes() will be the same for all three classes. If we create these classes avoiding inheritance then we have to write all of these functions in each of the three classes as shown below figure:



You can clearly see that the above process results in duplication of the same code 3 times. This increases the chances of error and data redundancy. To avoid this type of situation, inheritance is used. If we create a class Vehicle and write these three functions in it and inherit the rest of the classes from the vehicle class, then we can simply avoid the duplication of data and increase re-usability. Look at the below diagram in which the three classes are inherited from vehicle class:



Using inheritance, we have to write the functions only one time instead of three times as we have inherited the rest of the three classes from the base class (Vehicle).

Implementing inheritance:

A derived class is specified by defining its relationship with the base class in addition to its own details. The general syntax of defining a derived class is as follows:

For creating a sub-class that is inherited from the base class we have to follow the below syntax.

Derived Classes: A Derived class is defined as the class derived from the base class.

Syntax:

```
class <derived_class_name> : <access-specifier> <base_class_name>
{
    //body
}
```

Where

class : keyword to create a new class

derived_class_name: name of the new class, which will inherit the base class

access-specifier: either of private, public or protected. If neither is specified, PRIVATE is taken as default

base-class-name : name of the base class

Note: A derived class doesn't inherit **access** to private data members. However, it does inherit a full parent object, which contains any private members which that class declares.

The colon indicates that the a-class name is derived from the base class name. The access specifier or the visibility mode is optional and, if present, may be public, private or protected. By default it is private. Visibility mode describes the status of derived features e.g.

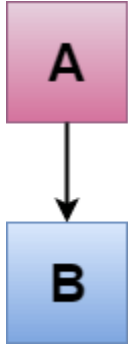
```
class xyz    //base class
{
    members of xyz
};
```

1. class ABC : private XYZ //private derivation
 { }
2. class ABC : public XYZ //public derivation
 { }
3. class ABC : protected XYZ //protected derivation
 { }
4. class ABC: XYZ //private derivation by default
 { }

In the inheritance, some of the base class data elements and member functions are inherited into the derived class. We can add our own data and member functions and thus extend the functionality of the base class. Inheritance, when used to modify and extend the capabilities of the existing classes, becomes a very powerful tool for incremental program development.

C++ Single Inheritance

Single inheritance is defined as the inheritance in which a derived class is inherited from the only one base class.



Where 'A' is the base class, and 'B' is the derived class.

C++ Single Level Inheritance Example: Inheriting Fields

When one class inherits another class, it is known as single level inheritance. Let's see the example of single level inheritance which inherits the fields only.

```
#include <iostream>
using namespace std;
class Account {
    public:
    float salary = 60000;
};
class Programmer: public Account {
    public:
    float bonus = 5000;
};
int main(void) {
    Programmer p1;
    cout<<"Salary: "<<p1.salary<<endl;
    cout<<"Bonus: "<<p1.bonus<<endl;
    return 0;
}
```

Output:

```
Salary: 60000  
Bonus: 5000
```

In the above example, Employee is the **base** class and Programmer is the **derived** class.

C++ Single Level Inheritance Example: Inheriting Methods

Let's see another example of inheritance in C++ which inherits methods only.

Let's see a simple example.

```
#include <iostream>
using namespace std;
class A
{
    int a = 4;
    int b = 5;
public:
    int mul()
    {
        int c = a*b;
        return c;
    }
};

class B : private A
{
public:
    void display()
    {
        int result = mul();
        cout << "Multiplication of a and b is : " << result << endl;
    }
};

int main()
{
```

```
B b;  
b.display();  
  
return 0;  
}
```

Output:

```
Multiplication of a and b is : 20
```

In the above example, class A is privately inherited. Therefore, the mul() function of class 'A' cannot be accessed by the object of class B. It can only be accessed by the member function of class B.

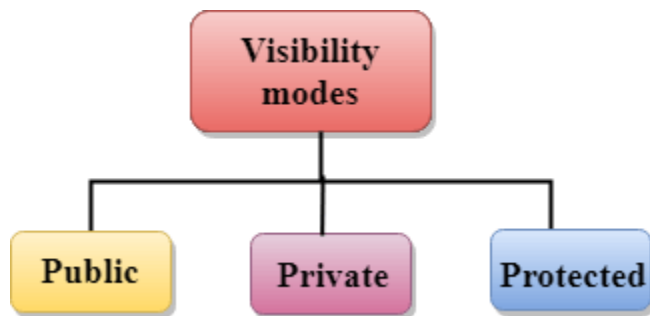
How to make a Private Member Inheritable

The private member is not inheritable. If we modify the visibility mode by making it public, but this takes away the advantage of data hiding.

C++ introduces a third visibility modifier, i.e., **protected**. The member which is declared as protected will be accessible to all the member functions within the class as well as the class immediately derived from it.

Visibility Mode or Access Specifier

Visibility modes can be classified into three categories:



- **Public:** When the member is declared as public, it is accessible to all the functions of the program.
- **Private:** When the member is declared as private, it is accessible within the class only.

- **Protected:** When the member is declared as protected, it is accessible within its own class as well as the class immediately derived from it.

Modes of Inheritance

1. **Public mode:** When the base class is publicly inherited, public members of the base class is publicly inherited, public members of the base class become public members of the derived class and therefore they are accessible to the objects of the derived class. In both the cases, the private members are not inherited and therefore, the private members of a base class will never become the members of its derived class.
2. **Protected mode:** When a base class is protected inherited by a derived class, public members of the base class can only be accessed by the member functions of the derived class. Private members of base class are inaccessible to the objects of the derived class. If private members of base class are to be inherited to derived class then declare them as protected.
3. **Private mode:** When a base class is privately inherited by a derived class, public members of the base class can only be accessed by the member functions of the derived class. Private members of base class are inaccessible to the objects of the derived class.

Note : The private members in the base class cannot be directly accessed in the derived class, while protected members can be directly accessed. For example, Classes B, C and D all contain the variables x, y and z in below example. It is just question of access.

// C++ Implementation to show that a derived class
// doesn't inherit access to private data members.

// However, it does inherit a full parent object

```
class A
{
public:
    int x;
protected:
    int y;
private:
    int z;
};
```

```
class B : public A
{
    // x is public
    // y is protected
    // z is not accessible from B
};
```

```
class C : protected A
```

```

{
    // x is protected
    // y is protected
    // z is not accessible from C
};

class D : private A    // 'private' is default for classes
{
    // x is private
    // y is private
    // z is not accessible from D
};

```

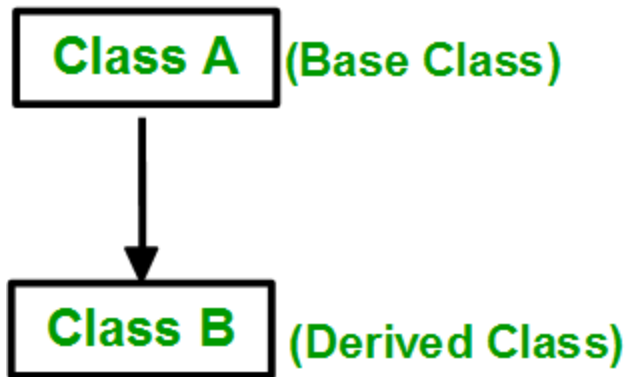
The below table summarizes the above three modes and shows the access specifier of the members of base class in the sub class when derived in public, protected and private modes:

Base class member access specifier	Type of Inheritance		
	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not accessible (Hidden)	Not accessible (Hidden)	Not accessible (Hidden)

Types of Inheritance in C++

1. Single inheritance
2. Multilevel inheritance
3. Multiple inheritance
4. Hierarchical inheritance
5. Hybrid inheritance

1. Single Inheritance: In single inheritance, a class is allowed to inherit from only one class. i.e. one sub class is inherited by one base class only.



Syntax:

```
class subclass_name : access_mode base_class
{
    //body of subclass
};
```

```
// C++ program to explain
// Single inheritance
#include <iostream>
```

```
class worker
{
    int age;
    char name [10];
public:
    void get ( );
    void show();
};
```

```
void worker :: get ( )
{
    cout << name;
    cout<< age;
}
void worker :: show ( )
{
    cout <<"In My Name is : " <<name<<"In My age is: "<<age;
}
```

```
class manager :: public worker //derived class (publicly)
{
    int now;
```

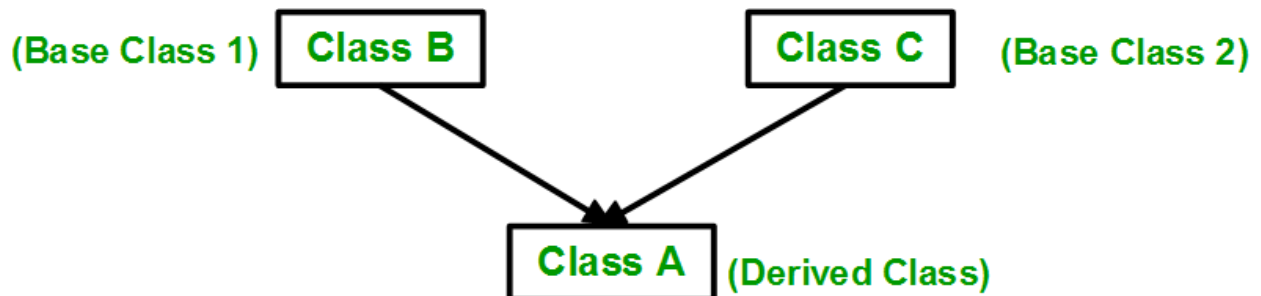
```

    public:
    void get ( ) ;
    void show ( ) ;
};
void manager : : get ( )
{ worker : : get ( ) ; //the calling of base class input fn.
  cout << "number of workers under you";
  cin >> now;
  cin>>name>>age; // ( if they were public )
}
void manager :: show ( )
{
  worker :: show ( ) ; //calling of base class o/p fn.
  cout <<< now;
}
Int main ( )
{
  worker W1;
  manager M1;
  M1 .get ( ) ;
  M1.show ( ) ;
  Return 0;
}

```

2. Multiple Inheritance:

Multiple Inheritance is a feature of C++ where a class can inherit from more than one classes. i.e one **sub class** is inherited from more than one **base classes**.



Syntax:

```

class subclass_name : access_mode base_class1, access_mode base_class2, ....
{
  //body of subclass
};

```

Here, the number of base classes will be separated by a comma (,) and access mode for every base class must be specified.

```
// C++ program to explain
// multiple inheritance
#include <iostream>
using namespace std;

// first base class
class Vehicle {
public:
    Vehicle()
    {
        cout << "This is a Vehicle" << endl;
    }
};

// second base class
class FourWheeler {
public:
    FourWheeler()
    {
        cout << "This is a 4 wheeler Vehicle" << endl;
    }
};

// sub class derived from two base classes
class Car: public Vehicle, public FourWheeler {

};

// main function
int main()
{
    // creating object of sub class will
    // invoke the constructor of base classes
    Car obj;
    return 0;
}
```

Output

This is a Vehicle
This is a 4 wheeler Vehicle

Ambiguity in Multiple Inheritance

In multiple inheritance, there may be possibility that a class may inherit member functions with same name from two or more base classes and the derived class may not have functions with same name as those of its base classes. If the object of the derived class need to access one of the same named member function of the base classes then it result in ambiguity as it is not clear to the compiler which base's class member function should be invoked. The ambiguity simply means the state when the compiler confused.

```
class a
{
private:
.....
public:
void abc() {}
.....
.....
};
class b
{
private:
.....
public:
void abc() {}
.....
.....
};
class c : public a , public b
{
private:
.....
public:
.....
.....
};
void main()
{
c obj;
obj.abc();//Error
.....
}
```

In this example, the two base classes **a** and **b** have function of same name **abc()** which are inherited to the derived class **c**. When the object of the class **c** is created and call the function **abc()** then the compiler is confused which base's class function is called by the compiler.

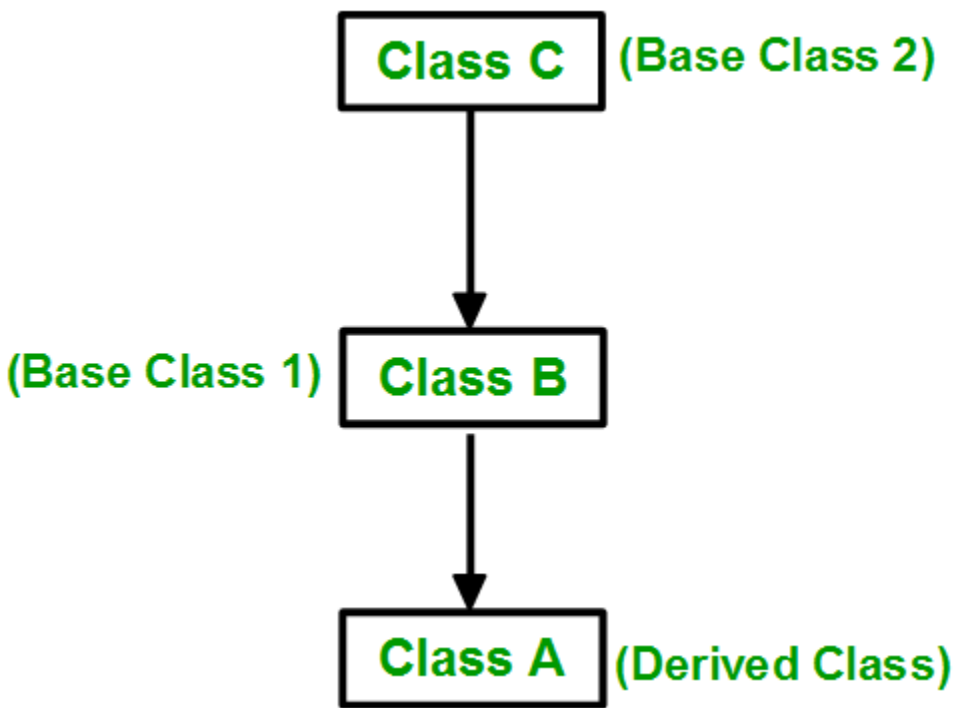
Solution of ambiguity in multiple inheritance: The ambiguity can be resolved by using the scope resolution operator to specify the class in which the member function lies as given below:

obj.a :: abc();

This statement invoke the function name abc() which are lies in base class a.

3. Multilevel Inheritance:

In this type of inheritance, a derived class is created from another derived class.



```
// C++ program to implement
// Multilevel Inheritance
#include <iostream>
using namespace std;

// base class
class Vehicle
{
public:
    Vehicle()
    {
        cout << "This is a Vehicle" << endl;
    }
};
```

```

// first sub_class derived from class vehicle
class fourWheeler: public Vehicle
{ public:
    fourWheeler()
    {
        cout<<"Objects with 4 wheels are vehicles"<<endl;
    }
};
// sub class derived from the derived base class fourWheeler
class Car: public fourWheeler{
public:
    Car()
    {
        cout<<"Car has 4 Wheels"<<endl;
    }
};

// main function
int main()
{
    //creating object of sub class will
    //invoke the constructor of base classes
    Car obj;
    return 0;
}

```

Output

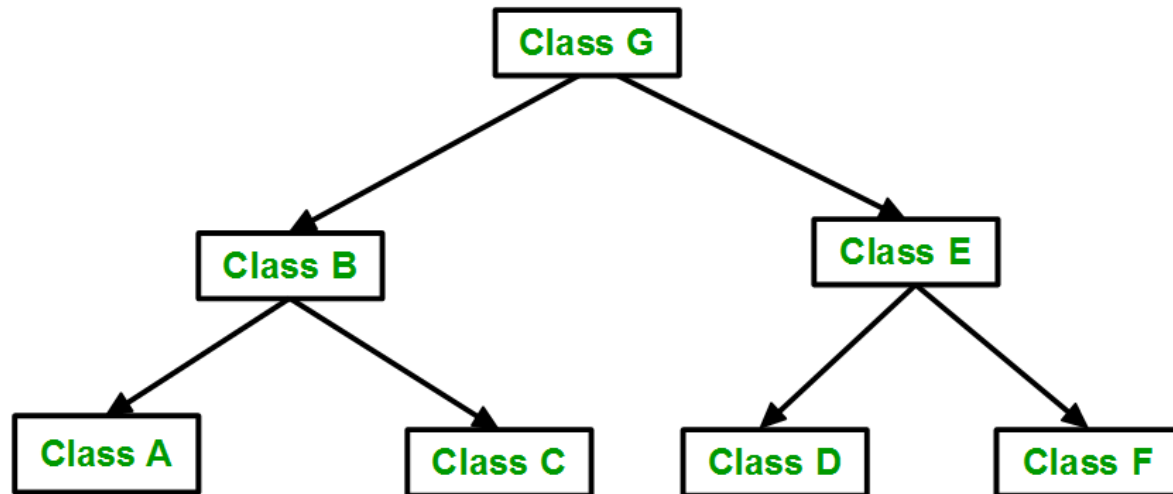
```

This is a Vehicle
Objects with 4 wheels are vehicles
Car has 4 Wheels

```

4. Hierarchical Inheritance:

In this type of inheritance, more than one sub class is inherited from a single base class. i.e. more than one derived class is created from a single base class.



```
// C++ program to implement
// Hierarchical Inheritance
#include <iostream>
using namespace std;

// base class
class Vehicle
{
public:
    Vehicle()
    {
        cout << "This is a Vehicle" << endl;
    }
};

// first sub class
class Car: public Vehicle
{
};

// second sub class
class Bus: public Vehicle
{
};

// main function
int main()
{
    // creating object of sub class will
```

```

    // invoke the constructor of base class
    Car obj1;
    Bus obj2;
    return 0;
}

```

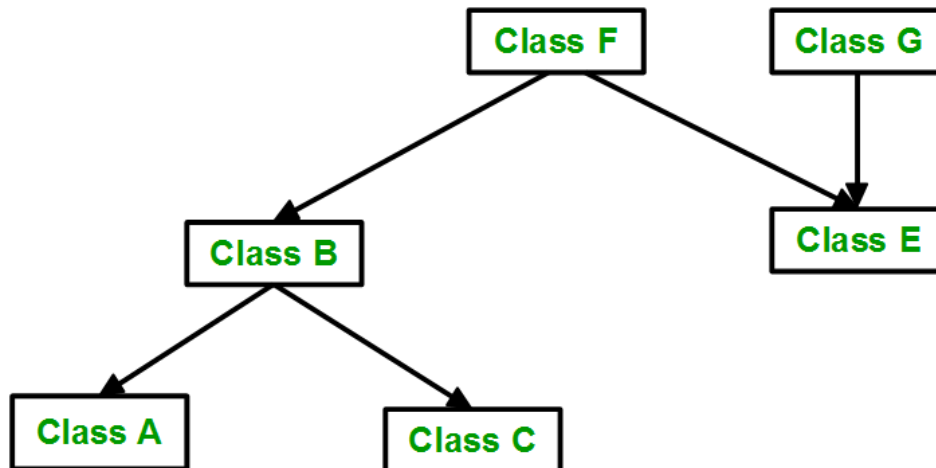
Output

This is a Vehicle
This is a Vehicle

5. Hybrid (Virtual) Inheritance:

Hybrid Inheritance is implemented by combining more than one type of inheritance. For example: Combining Hierarchical inheritance and Multiple Inheritance.

Below image shows the combination of hierarchical and multiple inheritance:



// C++ program for Hybrid Inheritance

```

#include <iostream>
using namespace std;

```

```

// base class
class Vehicle
{
public:
    Vehicle()
    {
        cout << "This is a Vehicle" << endl;
    }
}

```



```

};

//base class
class Fare
{
    public:
    Fare()
    {
        cout<<"Fare of Vehicle\n";
    }
};

// first sub class
class Car: public Vehicle
{

};

// second sub class
class Bus: public Vehicle, public Fare
{

};

// main function
int main()
{
    // creating object of sub class will
    // invoke the constructor of base class
    Bus obj2;
    return 0;
}

```

Output

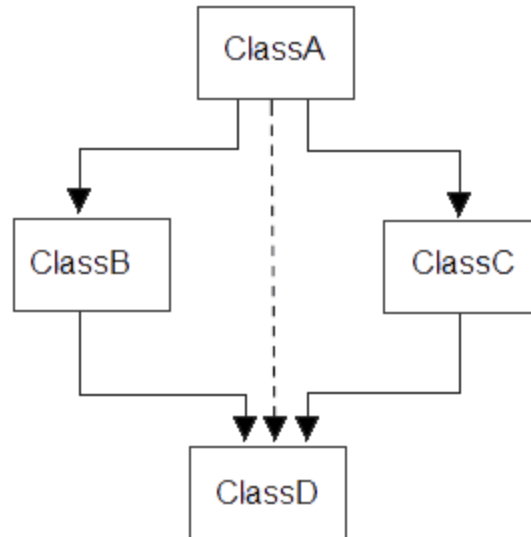
```

This is a Vehicle
Fare of Vehicle

```

6. A special case of hybrid inheritance : Multipath inheritance:

A derived class with two base classes and these two base classes have one common base class is called multipath inheritance. An ambiguity can arise in this type of inheritance.



/ C++ program demonstrating ambiguity in Multipath
// Inheritance

```
#include <conio.h>
#include <iostream.h>
class ClassA {
public:
    int a;
};

class ClassB : public ClassA {
public:
    int b;
};

class ClassC : public ClassA {
public:
    int c;
};

class ClassD : public ClassB, public ClassC {
public:
    int d;
};

void main()
{
    ClassD obj;

    // obj.a = 10;           //Statement 1, Error
```

```

// obj.a = 100;           //Statement 2, Error

obj.ClassB::a = 10; // Statement 3
obj.ClassC::a = 100; // Statement 4

obj.b = 20;
obj.c = 30;
obj.d = 40;

cout << "\n A from ClassB : " << obj.ClassB::a;
cout << "\n A from ClassC : " << obj.ClassC::a;

cout << "\n B : " << obj.b;
cout << "\n C : " << obj.c;
cout << "\n D : " << obj.d;
}

```

Output:

```

A from ClassB : 10
A from ClassC : 100
B : 20
C : 30
D : 40

```

In the above example, both ClassB & ClassC inherit ClassA, they both have single copy of ClassA. However ClassD inherit both ClassB & ClassC, therefore ClassD have two copies of ClassA, one from ClassB and another from ClassC.

If we need to access the data member a of ClassA through the object of ClassD, we must specify the path from which a will be accessed, whether it is from ClassB or ClassC, bco'z compiler can't differentiate between two copies of ClassA in ClassD.

There are 2 ways to avoid this ambiguity:

Avoiding ambiguity using scope resolution operator:

Using scope resolution operator we can manually specify the path from which data member a will be accessed, as shown in statement 3 and 4, in the above example.

```

obj.ClassB::a = 10;    //Statement 3
obj.ClassC::a = 100;   //Statement 4

```

Note : Still, there are two copies of ClassA in ClassD.

Avoiding ambiguity using virtual base class:

```
#include<iostream.h>
#include<conio.h>

class ClassA
{
    public:
    int a;
};

class ClassB : virtual public ClassA
{
    public:
    int b;
};

class ClassC : virtual public ClassA
{
    public:
    int c;
};

class ClassD : public ClassB, public ClassC
{
    public:
    int d;
};

void main()
{
    ClassD obj;

    obj.a = 10;    //Statement 3
    obj.a = 100;   //Statement 4

    obj.b = 20;
    obj.c = 30;
    obj.d = 40;

    cout<< "\n A : "<< obj.a;
    cout<< "\n B : "<< obj.b;
    cout<< "\n C : "<< obj.c;
    cout<< "\n D : "<< obj.d;
```

```
}
```

Output:

```
A : 100  
B : 20  
C : 30  
D : 40
```

According to the above example, ClassD has only one copy of ClassA, therefore, statement 4 will overwrite the value of a, given at statement 3.

C++ Aggregation (HAS-A Relationship)

In C++, aggregation is a process in which one class defines another class as any entity reference. It is another way to reuse the class. It is a form of association that represents HAS-A relationship.

Let's see an example of aggregation where Employee class has the reference of Address class as data member. In such way, it can reuse the members of Address class.

```
#include <iostream>  
using namespace std;  
class Address {  
    public:  
    string addressLine, city, state;  
    Address(string addressLine, string city, string state)  
    {  
        this->addressLine = addressLine;  
        this->city = city;  
        this->state = state;  
    }  
};  
class Employee  
{  
    private:  
    Address * address; //Employee HAS-A Address  
    public:  
    int id;  
    string name;  
    Employee(int id, string name, Address* address)
```

```

{
    this->id = id;
    this->name = name;
    this->address = address;
}
void display()
{
    cout<<id <<" "<<name<<" "<<
    address->addressLine<<" "<< address->city<<" "<<address->state<<endl;
}
};
int main(void) {
    Address a1= Address("C-146, Sec-15","Noida","UP");
    Employee e1 = Employee(101,"Nakul",&a1);
    e1.display();
    return 0;
}

```

Output:

```
101 Nakul C-146, Sec-15 Noida UP
```

Order of Constructor/ Destructor Call in C++

Whenever we create an object of a class, the default constructor of that class is invoked automatically to initialize the members of the class.

If we inherit a class from another class and create an object of the derived class, it is clear that the default constructor of the derived class will be invoked but before that the default constructor of all of the base classes will be invoke, i.e the order of invocation is that the base class's default constructor will be invoked first and then the derived class's default constructor will be invoked.

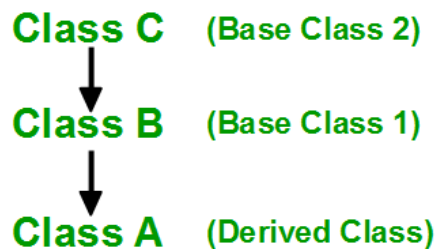
Why the base class's constructor is called on creating an object of derived class?

To understand this you will have to recall your knowledge on inheritance. What happens when a class is inherited from other? The data members and member functions of base class comes automatically in derived class based on the access specifier but the definition of these members

exists in base class only. So when we create an object of derived class, all of the members of derived class must be initialized but the inherited members in derived class can only be initialized by the base class's constructor as the definition of these members exists in base class only. This is why the constructor of **base class is called first to initialize all the inherited members.**

Order of constructor and Destructor call for a given order of Inheritance

Order of Inheritance



Order of Constructor Call

1. **C()** (Class C's Constructor)
2. **B()** (Class B's Constructor)
3. **A()** (Class A's Constructor)

Order of Destructor Call

1. **~A()** (Class A's Destructor)
2. **~B()** (Class B's Destructor)
3. **~C()** (Class C's Destructor)